

DEEP NEURAL NETWORKS (AIML ZG511) · SESSION 7

Convolutional Neural Networks

From raw pixels to learned features — understanding CNNs from the ground up

7th June 2026 | BITS MTech Programme | Module #6

How to use this companion. Each section starts in plain words. Then comes the analogy. Next the math is built step by step. Then a fully worked example with real numbers. Finally a key takeaway. Boxes are colour-coded: **blue = intuition**, **amber = everyday picture**, **green = worked example**, **red = watch out**, **purple = key takeaway**.

1 Image Classification and Its Challenges

Image classification means looking at a picture and deciding what is in it. For a human, this is easy. For a computer, it is hard.

The intuition

A digital image is just a grid of numbers. Each cell in the grid is a *pixel*, (picture element). Each pixel holds a brightness value between 0 (black) and 255 (white). A colour image has three such grids: one for Red, one for Green, one for Blue (RGB).

Why is this hard for a machine? The size grows fast. A small 50×50 image has $50 \times 50 = 2,500$ numbers. A standard HD image at 1920×1080 has about two million numbers. Each pixel becomes an input to the network.

★ Everyday picture

Think of a photo as a mosaic of tiny coloured tiles. You see the full picture at once. A computer has to read every single tile as a separate number and then figure out the picture from that list.

Three key challenges make this hard:

- **Viewpoint change.** A cat looks different from the side than from the front. The pixel values change completely, but the object is the same.
- **Translation invariance.** A cat in the top-left corner of an image has different pixel positions than a cat in the bottom-right corner. A good classifier must recognise the cat wherever it sits.
- **Scale invariance.** A small cat and a large cat should both be recognised. Shrinking an image changes the pixel values but not the object.

Naive approach and why it fails. A fully connected network treats every pixel as a separate input. For a 224×224 colour image, that is $224 \times 224 \times 3 = 150,528$ inputs. The first hidden layer alone would need millions of weights. This is too slow, too big, and ignores the fact that

pixels near each other are related.

✓ Key takeaway

An image is a grid of numbers. Two key ideas make classification tractable: (1) nearby pixels are related (*locality*), and (2) the same pattern can appear anywhere (*translation invariance*). The architecture we build must exploit both.

2 What is a Convolutional Neural Network?

A *Convolutional Neural Network* (CNN) is a type of deep neural network built specially for grid data, like images. Instead of connecting every neuron to every pixel, a CNN uses small, reusable *filters* that slide across the image.

The intuition

A CNN is inspired by the human visual cortex. Simple cells in the cortex respond to edges at specific angles. Complex cells combine those to see corners and curves. Deeper regions combine those into faces and objects. A CNN does the same: early layers detect simple patterns (edges), middle layers combine them (shapes), and deep layers combine shapes into objects.

★ Everyday picture

Imagine you are looking for your keys in a messy room. You do not scan every square centimetre of the whole room at once. You look in small patches, one at a time, and check whether the key shape is there. That is what a filter does in a CNN: it slides across the image in small patches and checks for a specific pattern.

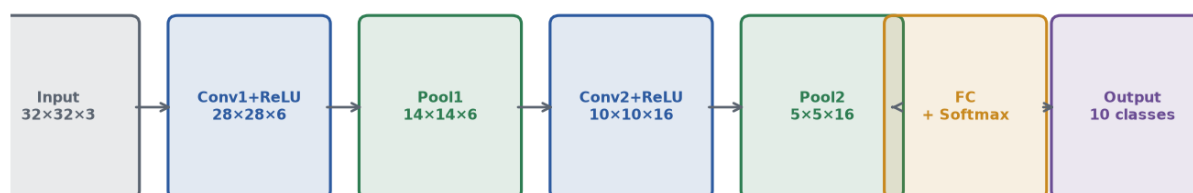
Three core principles make CNNs work:

1. **Local connectivity** (*sparse interactions*). Each neuron connects only to a small local patch of the input, called its *receptive field*. This saves parameters and captures the fact that nearby pixels belong together.
2. **Parameter sharing**. The same filter is applied at every position in the image. One set of weights learns one kind of pattern (say, a vertical edge) and finds it everywhere. This dramatically cuts the number of parameters.
3. **Equivariant representations**. If you shift the cat two pixels to the right, the feature map shifts by the same amount. The network's response moves with the object, which is exactly what we want.

✓ Key takeaway

CNNs replace the “every neuron sees everything” rule with “every neuron sees a small patch, but the same filter is reused everywhere.” This makes CNNs fast, small, and naturally good at finding patterns wherever they appear.

CNN Architecture: Input → Conv+Pool Blocks → Flatten → FC → Softmax



CNN high-level flow: input image goes through alternating Conv+Pool layers that extract features, then a

Fully Connected layer that makes the final class decision.

3 The Convolution Operation

The *convolution operation* is the heart of a CNN. It takes a small matrix called a *kernel* (or *filter*) and slides it across the input image, computing a dot product at each position.

The intuition

Think of a template. The kernel is a small template that says “this is what an edge looks like.” You slide it across the image. Where the template matches well, the dot product is large and positive. Where it does not match, the result is small or negative. The output of this sliding process is a new grid called a *feature map*.

The math. Let the input image be I (a matrix) and the kernel be K (a smaller matrix of size $k \times k$). For a 3×3 kernel at position (i, j) , the output value is:

$$O(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I(i+m, j+n) \cdot K(m, n)$$

We slide this window across the whole image. Each position gives one output number. The full set of output numbers forms the feature map.

Output size formula. For an $H \times W$ input, a $k \times k$ kernel, padding p , and stride s , the output size is:

$$H_{\text{out}} = \left\lfloor \frac{H - k + 2p}{s} \right\rfloor + 1, \quad W_{\text{out}} = \left\lfloor \frac{W - k + 2p}{s} \right\rfloor + 1$$

Worked example: Convolution on a 6×6 image with a 3×3 kernel

We have a 6×6 input image and a 3×3 kernel, stride $s = 1$, no padding ($p = 0$).

Step 1. Work out the output size.

$$H_{\text{out}} = \frac{6 - 3 + 0}{1} + 1 = 4, \quad W_{\text{out}} = 4.$$

The output feature map is 4×4 .

Step 2. Compute the top-left entry. Place the kernel at the top-left corner of the image (rows 1–3, cols 1–3). Multiply each kernel value with the corresponding image pixel, then add all nine products. Suppose this gives $O(0, 0) = 3$.

Step 3. Slide right by one pixel. Now the kernel covers rows 1–3, cols 2–4. Compute the nine products and sum: say $O(0, 1) = -1$.

Step 4. Continue sliding. Slide until the kernel reaches the right edge, then jump to the next row. After all 16 positions, we have the full 4×4 feature map:

$$\begin{pmatrix} 3 & -1 & -3 & -1 \\ -3 & -1 & 0 & -3 \\ -3 & -2 & 0 & 1 \\ 3 & -2 & -2 & -1 \end{pmatrix}$$

Step 5. Sense-check. Output size 4×4 : does the formula agree? $\frac{6-3}{1} + 1 = 4$. Yes.

Each entry in the feature map tells us how much the kernel’s pattern was present at that location in the input.



Convolution operation: the 3×3 kernel slides across the 6×6 input (arrows show direction) and builds a 4×4 feature map. Bright positions indicate strong pattern matches.

⚠ Watch out

The kernel must fully fit inside the image. You cannot place it so part of it hangs off the edge (unless you add padding). This is why the output is smaller than the input when no padding is used.

4 Padding and Stride

Two *hyperparameters* (settings we choose before training) control the convolution: *padding* and *stride*.

4.1 Padding

Padding adds extra pixels around the border of the image, usually zeros. There are three common types:

- **Valid padding** (no padding): the output is smaller than the input. Output size: $H - k + 1$.
- **Same padding**: we add enough zeros so the output is the same size as the input (assuming $s = 1$). For a $k \times k$ kernel: padding = $\frac{k-1}{2}$ (on each side).
- **Full padding**: we pad so every pixel is visited the maximum number of times. Rarely used in deep learning.

The intuition

Without padding, every convolution layer shrinks the image. After many layers, the feature maps become tiny. Same padding stops this shrinkage and also gives the pixels at the border an equal chance to contribute.

Worked example: Padding calculation for a 3×3 kernel

We want the output to have the same spatial size as a 6×6 input.

Step 1. Kernel size. $k = 3$.

Step 2. Required padding. $p = \frac{k-1}{2} = \frac{3-1}{2} = 1$. Add 1 pixel of zeros on each side. The padded image is 8×8 .

Step 3. Check output size. $H_{\text{out}} = \frac{6-3+2 \times 1}{1} + 1 = 6$. Same as input.

4.2 Stride

Stride s is how many pixels the kernel jumps at each step. Normally $s = 1$ (move one pixel). With $s = 2$, the kernel jumps two pixels each step. This gives a smaller output, which reduces computation.

Worked example: Stride-2 convolution on a 6×6 input

Input: 6×6 . Kernel: 3×3 . Stride: $s = 2$. No padding.

Step 1. Output size.

$$H_{\text{out}} = \left\lfloor \frac{6-3}{2} \right\rfloor + 1 = \lfloor 1.5 \rfloor + 1 = 2.$$

The output is 2×2 .

Step 2. Why smaller? With stride 2, the kernel only lands at positions 0, 2 (and then 0, 2 in the next row). That is 4 positions total.

Step 3. Trade-off. We lose spatial detail but save computation. Stride is often used instead of pooling to downsample.

Padding (left) vs. Stride (right) — controlling output size**Same Padding (p=1): 6×6 in → 6×6 out**

0	0	0	0	0	0	0	0
0							0
0							0
0							0
0							0
0							0
0							0
0	0	0	0	0	0	0	0

← padded border (zeros) →

Stride=3: 6×6 in → 2×2 out (green = kernel positions)

Left: same padding preserves spatial size. Right: stride-2 halves the spatial size. Both methods control the output resolution.

✓ Key takeaway

Padding controls whether the border pixels get equal treatment. Stride controls how fast we move across the image. Together they set the output feature-map size.

5 Dilated (Atrous) Convolution

A *dilated convolution* (also called *atrous convolution*) inserts spaces between kernel elements. The *dilation rate* r controls how many spaces. Standard convolution has $r = 1$ (no spaces).

The intuition

Imagine holding a 3-point ruler to measure a span. Normally the points are next to each other. With dilation rate 2, you hold the ruler with one empty slot between each point. The ruler still has 3 points (same parameters), but it covers a wider span of 5 slots.

Effective receptive field. For a $k \times k$ kernel with dilation rate r , the effective size is:

$$k_{\text{eff}} = k + (k - 1)(r - 1)$$

For $k = 3$, $r = 2$: $k_{\text{eff}} = 3 + 2 \times 1 = 5$. A 3×3 kernel with dilation 2 sees as wide a patch as a 5×5 kernel, but uses only 9 parameters (not 25).

Worked example: Effective receptive field for $k = 3$, $r = 3$

Step 1. Apply the formula. $k_{\text{eff}} = 3 + (3 - 1)(3 - 1) = 3 + 4 = 7$.

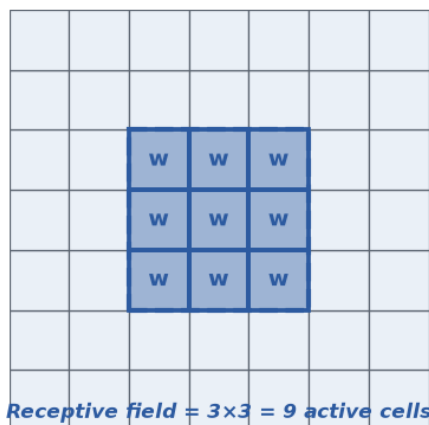
Step 2. Meaning. The kernel covers a 7×7 region with only $3 \times 3 = 9$ parameters.

Step 3. Compare. A standard 7×7 kernel would need 49 parameters. Dilation gives an $\approx 5\times$ saving.

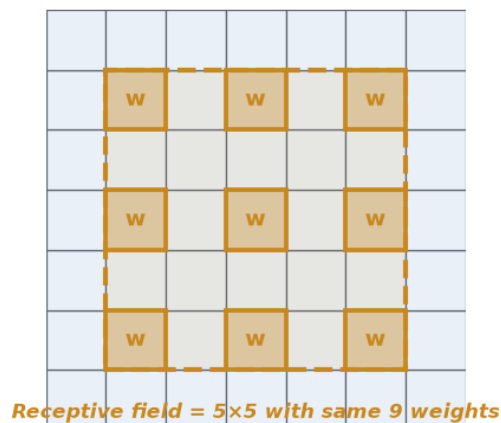
Where it is used. Semantic segmentation needs a large receptive field without losing resolution. Striding loses pixels. Pooling also loses pixels. Dilation keeps all pixels and widens the view. WaveNet uses dilated convolutions for long-range audio modelling.

Standard ($r=1$) vs Dilated ($r=2$) Convolution — same 9 weights, wider view

Standard ($r=1$): 3×3 kernel



Dilated ($r=2$): 3×3 kernel with gaps



Standard convolution (left, $r=1$, 3×3 field) vs. dilated convolution (right, $r=2$, 5×5 field). Same 9 weights; larger view.

⚠ Watch out

High dilation rates can cause *gridding artifacts* — gaps in the receptive field that produce a checkerboard pattern in outputs. Use a mix of dilation rates (e.g. 1, 2, 3, then repeat) to fill the gaps.

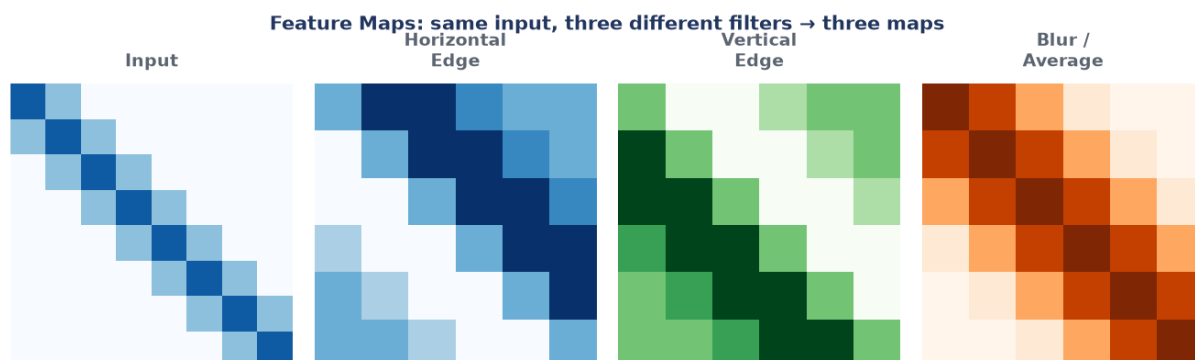
6 Feature Maps and Multiple Channels

6.1 Feature maps

When one filter slides across the input, the output is called a *feature map*. A feature map is a 2D grid that shows where in the image the filter's pattern was found. High values mean “strong match here.”

The intuition

A feature map is like a heat map. Bright spots show where the filter's pattern appears strongly. Dark spots mean that pattern is absent. An edge detector produces a bright feature map along edges and dark elsewhere.



Three filters (edge, curve, texture detectors) applied to the same input. Each filter gives a different feature map that highlights a different kind of pattern.

6.2 Multiple filters and 3D feature volumes

In practice we use many filters at once. Each filter produces one feature map. If we apply 64 filters to a 30×30 input, we get a volume of size $30 \times 30 \times 64$. The 64 depth slices are called *channels*.

Number of feature maps = number of filters.

6.3 Multi-channel input (RGB images)

A colour image has 3 channels (R, G, B). Each filter must also have 3 channels so it can process all three planes at once. For a filter of size 3×3 on a 3-channel input, the filter is actually $3 \times 3 \times 3 = 27$ numbers. One dot product (sum over all 27 products) gives one output number.

Parameter count for one filter:

$$\text{params per filter} = k \times k \times C_{\text{in}} + 1 \quad (\text{the } +1 \text{ is the bias})$$

Total parameters in a conv layer:

$$\text{Total params} = (k \times k \times C_{\text{in}} + 1) \times C_{\text{out}}$$

where C_{in} = input channels and C_{out} = number of filters (= output channels).

Worked example: Counting parameters in a conv layer (Example 1)

Input: RGB image ($224 \times 224 \times 3$). Kernel: 5×5 . Filters: 64. Bias: yes.

Step 1. Parameters per filter. $5 \times 5 \times 3 + 1 = 75 + 1 = 76$.

Step 2. Total parameters. $76 \times 64 = 4,864$.

Step 3. Note. The input spatial size (224×224) does NOT affect the parameter count. Parameters depend only on kernel size, input channels, and number of filters.

Worked example: Counting parameters in a conv layer (Example 2)

Input: feature map ($56 \times 56 \times 128$). Kernel: 3×3 . Filters: 256. Bias: yes.

Step 1. Parameters per filter. $3 \times 3 \times 128 + 1 = 1,152 + 1 = 1,153$.

Step 2. Total parameters. $1,153 \times 256 = 295,168$.

Step 3. Compare with Example 1. Same kernel size (3×3) but 128 input channels instead of 3. More input channels means more parameters per filter.

Typical channel progression in deep networks. Networks start with few channels and add more as spatial size shrinks:

$$\begin{aligned} 224 \times 224 \times 3 &\rightarrow 112 \times 112 \times 64 \rightarrow 56 \times 56 \times 128 \\ &\rightarrow 28 \times 28 \times 256 \rightarrow 14 \times 14 \times 512 \end{aligned}$$

Spatial size halves (via stride or pooling); channels double. This keeps total computation roughly constant across layers.

✓ Key takeaway

Parameters in a conv layer depend on kernel size and channel counts, not on spatial size. More filters = more output channels = more learned patterns.

7 The Receptive Field

The *receptive field* of a neuron is the region in the original input image that influences that neuron's value. A neuron in the first conv layer sees a small patch (e.g. 3×3). A neuron deeper in the network indirectly sees a much larger region.

★ Everyday picture

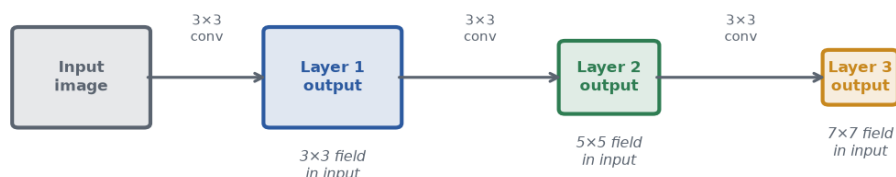
Think of Chinese whispers (telephone game). The first person hears one sentence. The tenth person has been influenced by many earlier messages combining to make the final one. A deep layer neuron is like the tenth person. Its value depends on a wide region of the original image. Each step only looked at a small patch.

Why it matters.

- Too small: cannot detect large objects.
- Too large: loses fine detail.
- Good design: receptive field matches the size of patterns the layer needs to detect.

How receptive field grows. With $k = 3$ and stride 1 at each layer, the receptive field grows by 2 per layer: Layer 1: 3×3 . Layer 2: 5×5 . Layer 3: 7×7 . Stride and pooling grow it faster.

Receptive Field Growth: each 3×3 conv adds 2 to the field seen in input



Receptive field growth: a neuron in layer 3 (red square) depends on a 7×7 patch of the original image, even though each conv layer used a 3×3 kernel.

✓ Key takeaway

Depth is what gives CNNs their wide view. Stacking small 3×3 kernels is more efficient than using one big kernel, and it builds up receptive field naturally.

8 Pooling Operations

Pooling layers reduce the spatial size of feature maps. They do not have learnable parameters — they apply a fixed rule (max or average) to small windows.

The intuition

Pooling is like a summary. You have a 4×4 block of values. Instead of keeping all 16, you keep one representative: either the maximum (most activated) or the average. This shrinks the map and also makes the detection slightly less sensitive to small shifts.

8.1 Max pooling

Take the **maximum** value in each pool window.

$$O(i, j) = \max_{0 \leq m < k, 0 \leq n < k} I(i \cdot s + m, j \cdot s + n)$$

Worked example: Max pooling: 4×4 input, $k = 2$, $s = 2$

Input:

$$\begin{pmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 7 & 8 & 3 & 0 \\ 2 & 1 & 4 & 9 \end{pmatrix}$$

Step 1. Top-left window (rows 0–1, cols 0–1): values 1, 3, 5, 6. Max = **6**.

Step 2. Top-right window (rows 0–1, cols 2–3): values 2, 4, 1, 2. Max = **4**.

Step 3. Bottom-left window (rows 2–3, cols 0–1): values 7, 8, 2, 1. Max = **8**.

Step 4. Bottom-right window (rows 2–3, cols 2–3): values 3, 0, 4, 9. Max = **9**.

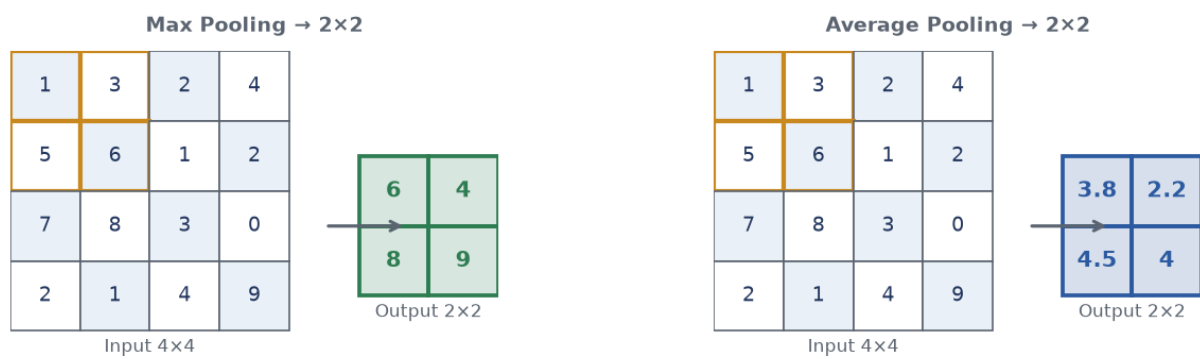
Step 5. Output (2×2): $\begin{pmatrix} 6 & 4 \\ 8 & 9 \end{pmatrix}$. Size formula: $\frac{4-2}{2} + 1 = 2$. Correct.

Max pooling keeps the strongest activation in each region.

8.2 Average pooling

Take the **average** of values in each window. Used in some final layers; smoother than max pooling.

Parameters in pooling layers: zero. Pooling has no weights. Only hyperparameters: pool size k and stride s .

Max Pooling (left) vs Average Pooling (right) — $k=2$, stride=2 on 4×4 input

Max pooling (left) and average pooling (right) both shrink spatial size. Max pooling preserves the strongest activation; average pooling gives a smoother summary.

✓ Key takeaway

Pooling reduces spatial size without trainable parameters. This saves computation and also gives modest translation invariance.

9 1×1 Convolution

A 1×1 convolution uses a kernel of size $1 \times 1 \times C_{\text{in}}$. It does not look at neighbouring pixels at all. It only mixes information across channels at each spatial position.

The intuition

Imagine each spatial position in a feature map has a list of C_{in} numbers (one per channel). A 1×1 convolution is just a *weighted sum* of those numbers: it combines them into a new list of C_{out} numbers. This lets you change the number of channels without changing spatial size.

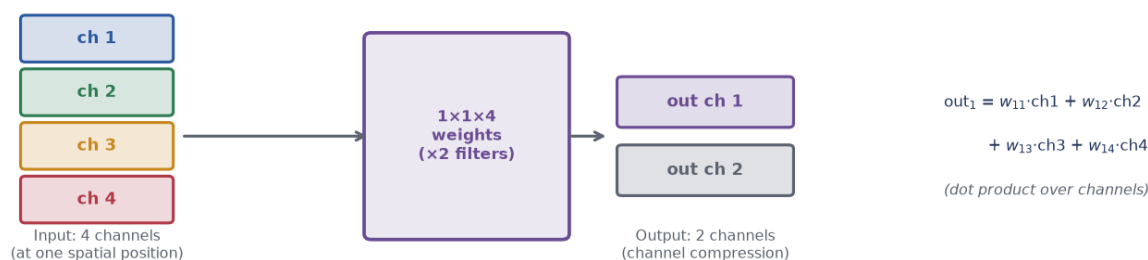
★ Everyday picture

Think of a chef mixing spice blends. Each position in the image has a rack of spices (channels). The 1×1 conv is a recipe that mixes them in a certain ratio. Applying 64 such recipes gives 64 new blends at every position.

Key uses.

- **Channel reduction (bottleneck).** Reduce from 256 channels to 64 channels before a costly 5×5 conv. This cuts parameters by 8x.
- **Channel expansion.** After reducing, expand back to 256.
- **Non-linear mixing.** Combined with ReLU, it adds non-linearity across channels (“network in network”).

1×1 Convolution: weighted mix of input channels $\rightarrow$ output channels



1×1 convolution mixes 64 input channels into 2 output channels at each spatial position. No spatial information is used — only channel mixing.

Worked example: Bottleneck: parameter savings with 1×1 conv

Standard path: 5×5 conv, 256 input channels, 256 output channels.

Step 1. Standard parameters. $(5 \times 5 \times 256 + 1) \times 256 \approx 1,638,656$.

Step 2. Bottleneck path. First a 1×1 conv: $256 \rightarrow 64$ channels: $(1 \times 1 \times 256 + 1) \times 64 = 16,448$. Then a 5×5 conv: $64 \rightarrow 64$: $(25 \times 64 + 1) \times 64 = 102,464$. Then a 1×1 conv: $64 \rightarrow 256$: $(64 + 1) \times 256 = 16,640$. Total: $16,448 + 102,464 + 16,640 = \mathbf{135,552}$.

Step 3. Saving. $\approx 12 \times$ fewer parameters.

Bottleneck design is used in ResNet-50/101/152 and Inception.

10 Activation Functions in CNNs

10.1 ReLU

ReLU (Rectified Linear Unit) is the default activation function for CNNs. It is applied element-wise after each conv layer.

$$\text{ReLU}(x) = \max(0, x)$$

If the input is positive, pass it through. If negative, output zero.

★ Everyday picture

A light switch that only turns on when voltage is positive, and stays off otherwise. Simple, fast, and it keeps the information that matters.

Why ReLU works well.

- **No vanishing gradient.** For positive inputs, the gradient is exactly 1. Unlike sigmoid (σ) or tanh, it does not squash gradients toward zero in deep networks.
- **Sparse.** Many neurons output 0, which means the network only passes forward the important activations. This is efficient.
- **Fast to compute.** Just a max operation — no expensive exponentials.

✗ Watch out

The *dying ReLU* problem: if the input to a neuron is always negative, the neuron always outputs 0 and its gradient is always 0. It can never recover. This is “dead” neuron. Using a small learning rate and careful initialisation reduces this risk. Leaky ReLU (output $0.01x$ for $x < 0$) also helps.

10.2 Softmax

Softmax turns a vector of raw scores into a probability distribution. It is used in the final output layer for multi-class classification.

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

Here z_i is the raw score for class i , and C is the total number of classes. The outputs sum to 1.

Worked example: Softmax on 3 classes

Raw scores: $z = [2.0, 1.0, 0.1]$.

Step 1. Exponentiate each score. $e^{2.0} = 7.389$, $e^{1.0} = 2.718$, $e^{0.1} = 1.105$.

Step 2. Sum the exponentials. $Z = 7.389 + 2.718 + 1.105 = 11.212$.

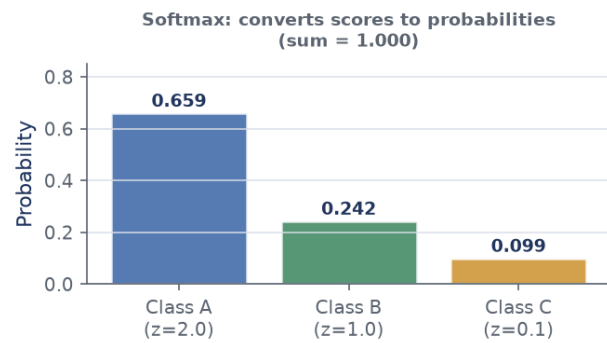
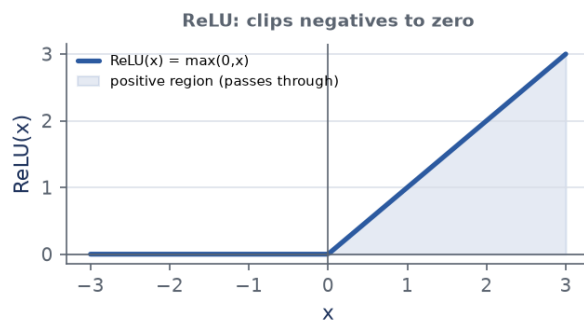
Step 3. Divide each by the sum.

$$p_1 = \frac{7.389}{11.212} = \mathbf{0.659}, \quad p_2 = \frac{2.718}{11.212} = \mathbf{0.242}, \quad p_3 = \frac{1.105}{11.212} = \mathbf{0.099}.$$

Step 4. Check. $0.659 + 0.242 + 0.099 = 1.000$. All probabilities lie between 0 and 1 and sum to 1.

Class 1 gets 65.9% probability, class 2 gets 24.2%, class 3 gets 9.9%.

Activation Functions: ReLU (left) and Softmax (right)



Left: ReLU clips negative values to zero (the hinge shape). Right: softmax squashes any three scores into three probabilities that sum to 1.

11 The Fully Connected Layer and Flattening

After several conv and pooling layers, we have a small but deep feature volume (e.g. $5 \times 5 \times 16$). A *Fully Connected* (FC) layer takes this volume and connects every value to every output neuron.

Flattening. First we reshape the volume into a 1D vector:

$$(5 \times 5 \times 16) \longrightarrow 400\text{-dimensional vector}$$

The FC layer.

$$\mathbf{y} = f(\mathbf{W}\mathbf{x} + \mathbf{b})$$

where $\mathbf{x} \in \mathbb{R}^n$ is the input vector, $\mathbf{W} \in \mathbb{R}^{m \times n}$ is the weight matrix, $\mathbf{b} \in \mathbb{R}^m$ is the bias vector, and $f(\cdot)$ is the activation. Output: $\mathbf{y} \in \mathbb{R}^m$.

Worked example: Parameter count in a fully connected layer

Flatten output: 400. FC layer to 120 neurons. Then FC to 10 classes.

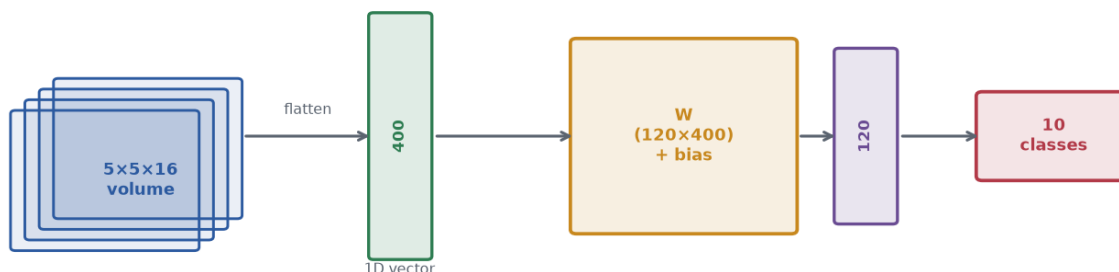
Step 1. First FC layer (400 → 120). Parameters: $400 \times 120 + 120 = 48,120$.

Step 2. Second FC layer (120 → 10). Parameters: $120 \times 10 + 10 = 1,210$.

Step 3. Total FC parameters. $48,120 + 1,210 = 49,330$.

FC layers dominate parameter counts. This is why they are often replaced by Global Average Pooling (GAP).

Flatten + Fully Connected Layer: 3D volume → 1D vector → class scores



Flattening reshapes the $5 \times 5 \times 16$ feature volume into a 400-element vector. The FC layer then multiplies by a 120×400 weight matrix to produce 120 output values.

⚠ Watch out

FC layers require a fixed input size. If the image is a different size, you get a different-length vector and the weights no longer match. Convolutional layers are size-agnostic; FC layers are not.

12 The Full CNN Architecture

The standard pattern is:

$$\text{INPUT} \rightarrow [\text{CONV} \rightarrow \text{ReLU} \rightarrow \text{POOL}]^{\times N} \rightarrow \text{Flatten} \rightarrow \text{FC} \rightarrow \text{Softmax}$$

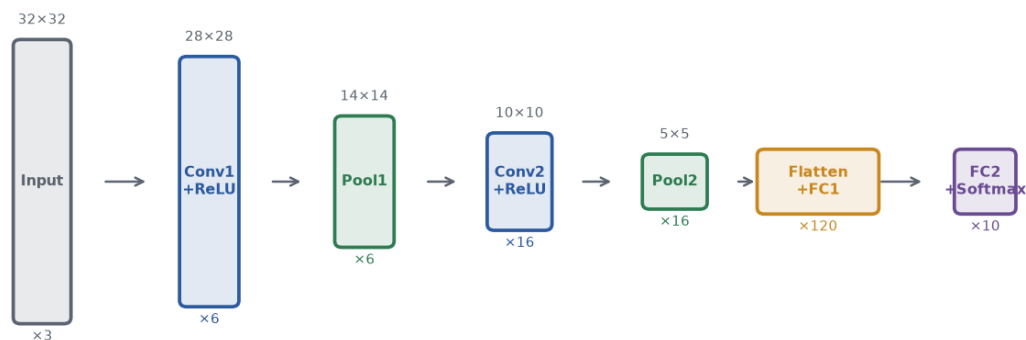
A concrete example (LeNet-style) for $32 \times 32 \times 3$ input, 10 classes:

Layer	Output size	Operation	Params
Input	$32 \times 32 \times 3$	—	0
Conv1 + ReLU	$28 \times 28 \times 6$	5×5 , 6 filters	456
Pool1 (Max)	$14 \times 14 \times 6$	2×2 , stride 2	0
Conv2 + ReLU	$10 \times 10 \times 16$	5×5 , 16 filters	2,416
Pool2 (Max)	$5 \times 5 \times 16$	2×2 , stride 2	0
Flatten	400	reshape	0
FC1 + ReLU	120	fully connected	48,120
FC2	10	fully connected	1,210
Softmax	10	probabilities	0
Total			52,202

The intuition

Early layers extract low-level features (edges, corners). Middle layers combine these into shapes and textures. The final FC layers combine everything to vote for a class. Each layer narrows down “what is here?” until the last layer gives a single answer.

LeNet-style CNN: spatial size shrinks, channels grow, FC decides class



Full CNN architecture: spatial size shrinks (Conv+Pool blocks), channel count grows, then FC layers collapse to class probabilities. Numbers above boxes show spatial size; numbers below show channels.

13 Learning Kernels via Backpropagation

The kernel weights are *not* hand-designed. They start random and are learned by the same backpropagation algorithm that trains FC networks.

The intuition

The network runs a forward pass: compute the loss. Then it runs backward: use the chain rule to find how much each kernel weight contributed to the loss. Then nudge the weight a tiny step in the direction that reduces the loss. After thousands of images, the kernels settle into useful patterns.

The gradient for a kernel weight $K(m, n)$ is:

$$\frac{\partial L}{\partial K(m, n)} = \sum_{i, j} \frac{\partial L}{\partial O(i, j)} \cdot I(i + m, j + n)$$

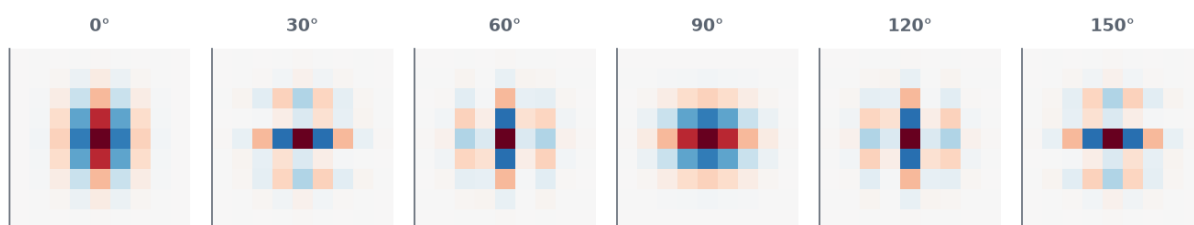
This sums the upstream gradient over every position where the kernel was applied. Parameter sharing means all applications of the kernel share the same update.

★ Everyday picture

Imagine every worker at a factory uses the same manual. If the factory makes an error, every worker's manual gets the same correction note. That is parameter sharing in backprop: the kernel update pools feedback from every position it was applied.

What do learned kernels look like? After training on images, first-layer kernels often look like Gabor filters: oriented edges at different angles, blobs, and colour selectors. This mirrors what V1 neurons in the human visual cortex respond to.

First-layer kernels after training: edge detectors at various angles



After training, kernels in the first layer look like edge detectors at different angles and frequencies. Each row is one filter; bright = positive weight, dark = negative weight.

14 Design Patterns for Deep CNNs

Modern CNNs use five recurring design patterns.

14.1 Pattern 1: Spatial Reduction + Channel Expansion

As we go deeper, halve the spatial size (stride-2 conv or pooling) and double the number of channels. This keeps the total amount of information (volume $H \times W \times C$) roughly constant across layers.

14.2 Pattern 2: Repetitive Building Blocks

Use a small standard module and repeat it. VGG uses *VGG blocks*: two or three 3×3 convolutions followed by a max pool. This is simpler to scale and easier to tune than arbitrary layer stacks.

14.3 Pattern 3: Skip / Residual Connections

ResNets add the input $\mathbf{x}$ directly to the output of two conv layers:

$$\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$$

where $F(\mathbf{x})$ is two conv layers and $\mathbf{x}$ is the shortcut.

The intuition

If the best thing to do is nothing (the output should equal the input), the layers only need to learn $F(\mathbf{x}) = 0$, which is easy. Without the skip connection, the layers would have to learn the identity function, which is hard for deep networks.

Why residual connections matter. They give gradients a direct path backward through the network. In a 100-layer network without skip connections, gradients shrink (vanish) before they reach early layers. Skip connections fix this.

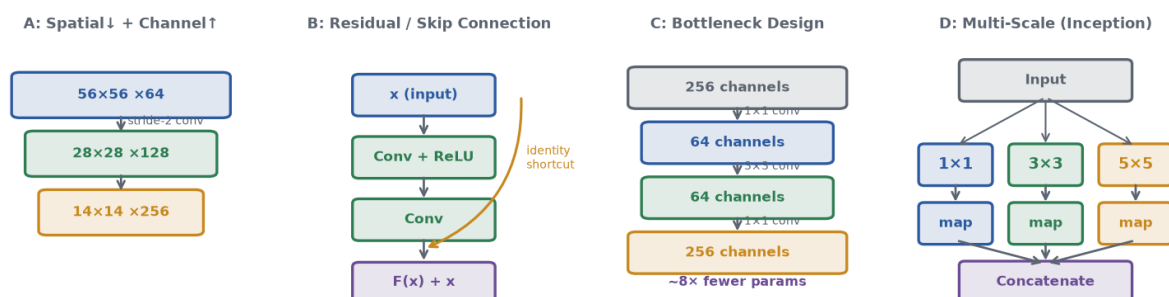
14.4 Pattern 4: Bottleneck Designs

Instead of one expensive 3×3 conv on many channels, use: 1×1 (reduce channels) $\rightarrow 3 \times 3$ (spatial conv on fewer channels) $\rightarrow 1 \times 1$ (restore channels). This cuts computation by roughly 8x with almost no accuracy loss.

14.5 Pattern 5: Multi-Scale Feature Extraction (Inception)

Process the input with 1×1 , 3×3 , 5×5 kernels in parallel, then concatenate all outputs along the channel dimension. The network learns which scale is most useful for each feature.

Four CNN Design Patterns



Five design patterns: (A) spatial reduction with channel expansion; (B) residual/skip connections with

identity shortcut; (C) bottleneck block; (D) Inception multi-scale module with parallel convolutions.

✓ **Key takeaway**

Modern architectures are not arbitrary layer stacks. They use proven patterns: skip connections for depth, bottlenecks for efficiency, and spatial reduction plus channel expansion to maintain information flow.

15 Choosing Architecture Depth and Width

Depth is the number of layers. *Width* is the number of channels per layer.

Guidelines for depth:

- Simple task (MNIST): 5–10 layers enough.
- ImageNet: 50–152 layers with skip connections.
- Small dataset (<10K samples): stay shallow to avoid over-fitting.
- Large dataset (>1M samples): go deep (50–200 layers).

Guidelines for width:

- Wider networks learn more diverse patterns at each layer.
- Wide ResNet uses fewer layers but much wider channels (up to 10× standard).

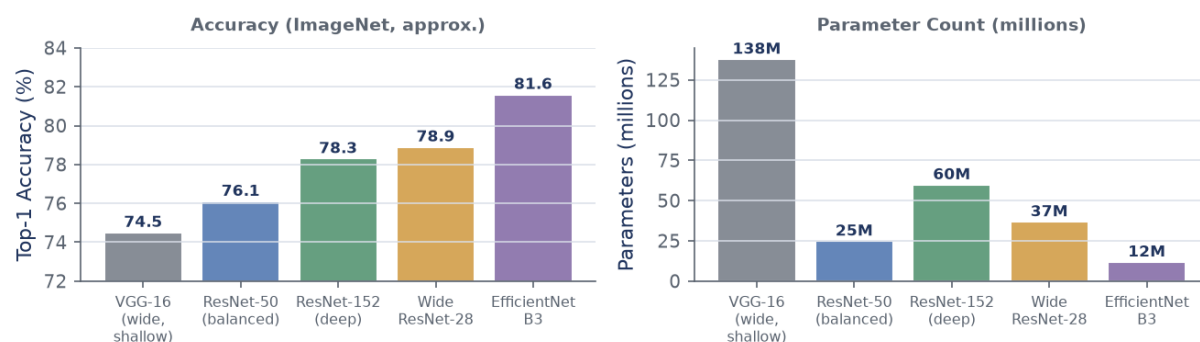
EfficientNet’s compound scaling. Scale all three dimensions (depth d , width w , resolution r) together:

$$d^\alpha \cdot w^\beta \cdot r^\gamma = 2$$

under a fixed computational budget. This achieves a better accuracy/efficiency trade-off than scaling any single dimension.

Practical decision framework:

Depth vs Width: top-1 accuracy and parameter count for common architectures



Depth vs. width trade-off: deeper networks build more complex hierarchies; wider networks learn more diverse patterns. EfficientNet scales both together for the best balance.

1. Start with a proven baseline (ResNet-50, EfficientNet).
2. If under-fitting: add depth or width.
3. If over-fitting: add regularisation (dropout, data augmentation) before cutting capacity.
4. If too slow: reduce width or use bottleneck blocks.
5. For mobile: use MobileNet or EfficientNet-B0.

✓ Key takeaway

Start shallow and widen/deepen only when needed. Use skip connections for any network deeper than ~ 10 layers. Scale depth, width, and resolution together for best results.

16 Putting It All Together: A Working Example

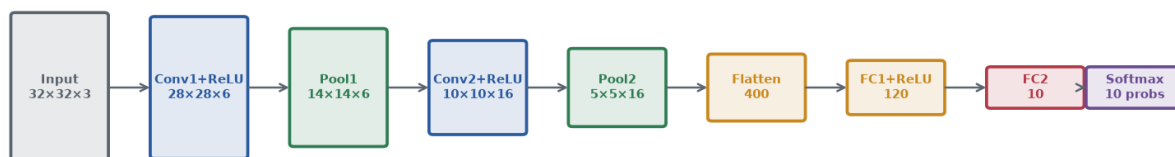
Let us trace a single forward pass through the LeNet-style network from Section 11 for a $32 \times 32 \times 3$ image with 10 classes.

Worked example: Full forward pass through a small CNN

- Step 1. Input.** A colour image: shape $32 \times 32 \times 3$. All pixel values are in $[0, 255]$; we normalise to $[0, 1]$.
- Step 2. Conv1 (5×5 , 6 filters, valid padding, stride 1).** Output: $\frac{32-5}{1} + 1 = 28$. Shape: $28 \times 28 \times 6$. Apply ReLU: set all negatives to zero.
- Step 3. Pool1 (max, 2×2 , stride 2).** Output: $\frac{28-2}{2} + 1 = 14$. Shape: $14 \times 14 \times 6$. No learnable parameters.
- Step 4. Conv2 (5×5 , 16 filters, valid padding, stride 1).** Output: $\frac{14-5}{1} + 1 = 10$. Shape: $10 \times 10 \times 16$. Apply ReLU.
- Step 5. Pool2 (max, 2×2 , stride 2).** Output: $\frac{10-2}{2} + 1 = 5$. Shape: $5 \times 5 \times 16$.
- Step 6. Flatten.** $5 \times 5 \times 16 = 400$. Shape: (400,).
- Step 7. FC1 ($400 \rightarrow 120$, ReLU).** Matrix multiply: $(120, 400) \cdot (400,) + (120,) = (120,)$. Apply ReLU.
- Step 8. FC2 ($120 \rightarrow 10$).** Output: 10 raw scores (one per class).
- Step 9. Softmax.** Convert 10 scores to 10 probabilities. The class with the highest probability is the prediction.

The total parameter count is 52,202 (see the table in Section 11). This is tiny compared to a fully connected approach, which would need $32 \times 32 \times 3 \times$ hidden size weights in the first layer alone.

Forward Pass: shapes through the LeNet-style CNN ($32 \times 32 \times 3$ input, 10 classes)



One forward pass traced: spatial size shrinks at each Pool layer, and the flattened vector feeds the FC layers to produce 10 class scores.